

Problem ID	Captured Flags	Steps
Nuclear Pasta	fsuCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}	1. Analyze the binary in a decompiler and identify the control-flow flattening (spaghetti code) obfuscation. 2. Trace the valid execution path using a debugger or symbolic execution tools like angr. 3. Identify the constraints and transformations applied to the user input along the execution path. 4. Solve these constraints to reconstruct the correct input flag.
Old School Gauntlet	fsuCTF{b3tt3r_7h4n_cry5t41_4rm0r}	1. Decompile the binary and locate the encrypted data blocks cross-referenced in main. 2. Identify the ptrace and timing-based anti-debugging checks designed to hide the payload. 3. Patch the binary's assembly to neutralize the anti-debugging conditional jumps. 4. Run the patched program in a debugger to dynamically extract the decrypted flag from memory.
Upper Management	fsuCTF{5ti11_b3tter_7h4n_73ch_supp0r7}	1. Decompile the stripped binary to locate the target byte array and the 38-character input length check. 2. Identify the character transformation loop and recognize

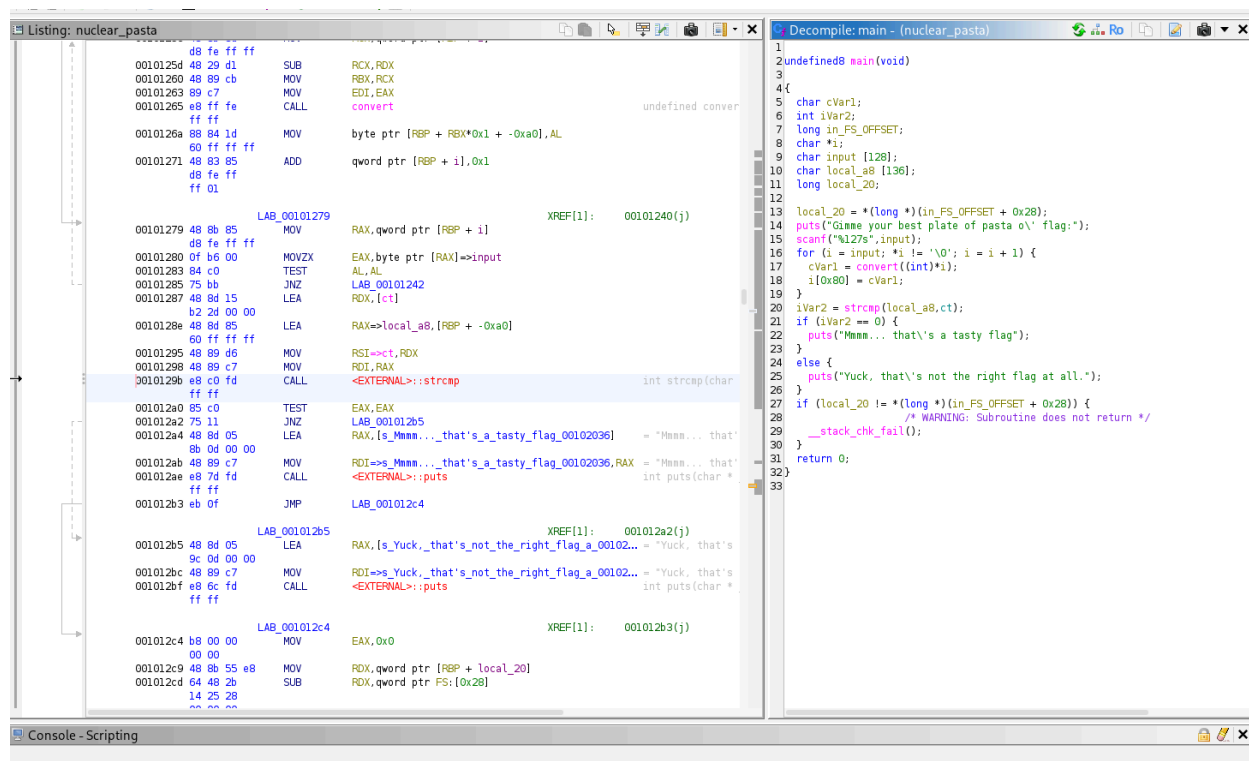
		the Mixed Boolean Arithmetic (MBA) obfuscation. 3. Write a Python script for GDB using starti to safely attach to the live, stripped process. 4. Brute-force the flag by dynamically calling the binary's internal ciphering function for every possible character.
--	--	---

Nuclear Pasta

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
└─$ ls
gauntlet  nuclear_pasta  upper_management

(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
└─$
```

First I download the binary and make it executable by adding the permissions using “chmod +x nuclear pasta.”



Then I open the file in ghidra and start looking at the main function. In the main function I found several things: The program prompts the user for a flag: `puts("Gimme your best plate of pasta o\` flag:");`. It reads up to 127 characters of input into a buffer using `scanf("%127s", input);`. It then enters a for loop that iterates over every character of your input string. Inside the loop, it passes the current character to a function called `convert()`. The converted character is stored into another buffer (which maps to `local_a8`). Finally, it calls `strcmp(local_a8, ct);` to compare the modified input against a global variable named `ct`. If `strcmp` returns 0 (meaning the strings match perfectly), it prints "Mmmm... that's a tasty flag". So we need to know what is inside `ct` and we need to know what the `convert` function does.

```

22     }
23     else {
24         local_1c = local_1c ^ 0xbb;
25     }
26 } while ((int)local_10 % (int)local_14 == 0);
27 return local_1c;
28 }

```

In the `convert` function I find the following which shows that there is an XOR encryption using 0xbb. This is now easily solvable:

Listing: nuclear_pasta				
00104040	dd	undefined1DDh	[0]	XREF[
00104041	c8	undefined1C8h	[1]	
00104042	ce	undefined1CEh	[2]	
00104043	f8	undefined1F8h	[3]	
00104044	ef	undefined1EFh	[4]	
00104045	fd	undefined1FDh	[5]	
00104046	c0	undefined1C0h	[6]	
00104047	c8	undefined1C8h	[7]	
00104048	cb	undefined1CBh	[8]	
00104049	8f	undefined18Fh	[9]	
0010404a	dc	undefined1DCh	[10]	
0010404b	d3	undefined1D3h	[11]	
0010404c	de	undefined1DEh	[12]	
0010404d	cf	undefined1CFh	[13]	
0010404e	cf	undefined1CFh	[14]	
0010404f	8a	undefined18Ah	[15]	
00104050	e4	undefined1E4h	[16]	
00104051	d8	undefined1D8h	[17]	
00104052	8b	undefined18Bh	[18]	
00104053	df	undefined1DFh	[19]	
00104054	88	undefined188h	[20]	
00104055	e4	undefined1E4h	[21]	
00104056	8a	undefined18Ah	[22]	
00104057	8e	undefined18Eh	[23]	
00104058	e4	undefined1E4h	[24]	
00104059	da	undefined1DAh	[25]	
0010405a	e4	undefined1E4h	[26]	
0010405b	df	undefined1DFh	[27]	
0010405c	88	undefined188h	[28]	
0010405d	dd	undefined1DDh	[29]	
0010405e	de	undefined1DEh	[30]	
0010405f	d5	undefined1D5h	[31]	
00104060	c8	undefined1C8h	[32]	
00104061	88	undefined188h	[33]	
00104062	e4	undefined1E4h	[34]	
00104063	d6	undefined1D6h	[35]	
00104064	de	undefined1DEh	[36]	
00104065	d8	undefined1D8h	[37]	
00104066	d3	undefined1D3h	[38]	
00104067	8f	undefined18Fh	[39]	
00104068	d5	undefined1D5h	[40]	
00104069	8a	undefined18Ah	[41]	
0010406a	c8	undefined1C8h	[42]	
0010406b	d6	undefined1D6h	[43]	
0010406c	c6	undefined1C6h	[44]	
0010406d	00	undefined100h	[45]	

I go to ct in memory and find what is written there which is the following bytes. Then I just show the screenshot of the bytes and the XOR encryption to Gemini and it solves it for me after a couple attempts:

If we XOR those with `0xbb` :

- `0xd6 ^ 0xbb = m`
- `0xde ^ 0xbb = e` (In my last message, I accidentally typed this as a '3'!)
- `0xd8 ^ 0xbb = c`
- `0xd3 ^ 0xbb = h`
- `0x8f ^ 0xbb = 4`
- `0xd5 ^ 0xbb = n`
- `0x8a ^ 0xbb = 1`
- `0xc8 ^ 0xbb = s`
- `0xd6 ^ 0xbb = m`

The word is `mech4n1sm` , not `m3ch4n1sm` .

Here is the actual, corrected flag:

```
f5uCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}
```

Flag: f5uCTF{sp4ghett1_c0d3_15_a_d3fens3_mech4n1sm}

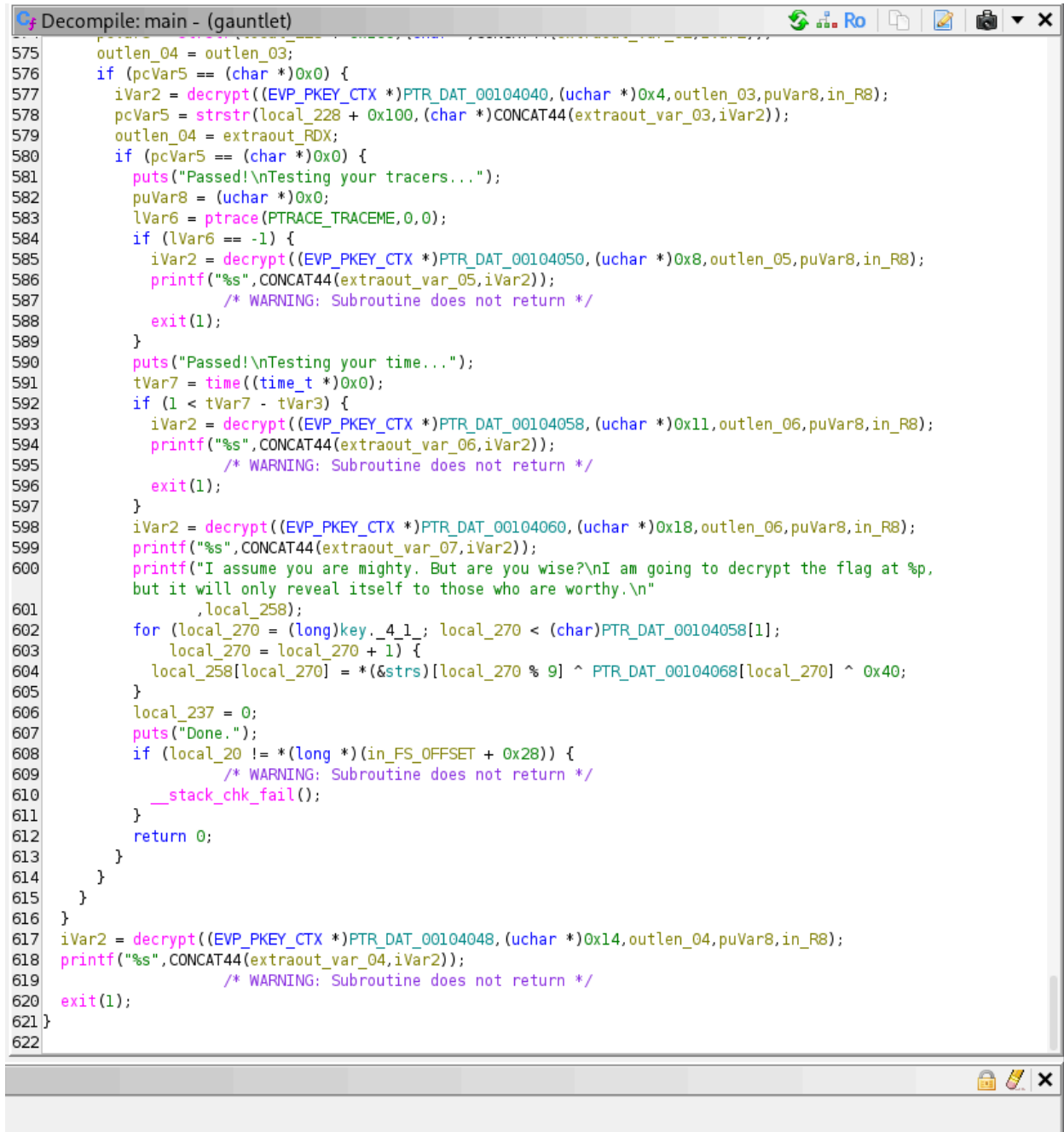
Old School Gauntlet

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$ ./gauntlet
Welcome, Warrior of the Amlodd!
Prove your might in... the gauntlet...
Testing for guardians...
Passed!
Testing your tracers...
Passed!
Testing your time...
This program is fine :)
I assume you are mighty. But are you wise?
I am going to decrypt the flag at 0x7ffffdd25af0, but it will only reveal itself to those who are worthy.
Done.

(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$
```

This output tells us the `gauntlet` program uses anti-debugging techniques, such as tracer detection and execution time checks, to prevent analysis while it secretly decrypts the flag into randomized stack

memory without ever printing it. To bypass these defenses, I choose to use static techniques utilizing ghidra.



```
Decompile: main - (gauntlet)
575 outlen_04 = outlen_03;
576 if (pcVar5 == (char *)0x0) {
577     iVar2 = decrypt((EVP_PKEY_CTX *)PTR_DAT_00104040, (uchar *)0x4, outlen_03, puVar8, in_R8);
578     pcVar5 = strstr(local_228 + 0x100, (char *)CONCAT44(extraout_var_03, iVar2));
579     outlen_04 = extraout_RDX;
580     if (pcVar5 == (char *)0x0) {
581         puts("Passed!\nTesting your tracers...");
582         puVar8 = (uchar *)0x0;
583         lVar6 = ptrace(PTRACE_TRACEME, 0, 0);
584         if (lVar6 == -1) {
585             iVar2 = decrypt((EVP_PKEY_CTX *)PTR_DAT_00104050, (uchar *)0x8, outlen_05, puVar8, in_R8);
586             printf("%s", CONCAT44(extraout_var_05, iVar2));
587             /* WARNING: Subroutine does not return */
588             exit(1);
589         }
590         puts("Passed!\nTesting your time...");
591         tVar7 = time((time_t *)0x0);
592         if (1 < tVar7 - tVar3) {
593             iVar2 = decrypt((EVP_PKEY_CTX *)PTR_DAT_00104058, (uchar *)0x11, outlen_06, puVar8, in_R8);
594             printf("%s", CONCAT44(extraout_var_06, iVar2));
595             /* WARNING: Subroutine does not return */
596             exit(1);
597         }
598         iVar2 = decrypt((EVP_PKEY_CTX *)PTR_DAT_00104060, (uchar *)0x18, outlen_06, puVar8, in_R8);
599         printf("%s", CONCAT44(extraout_var_07, iVar2));
600         printf("I assume you are mighty. But are you wise?\nI am going to decrypt the flag at %p,
        but it will only reveal itself to those who are worthy.\n"
        , local_258);
601         for (local_270 = (long)key._4_1; local_270 < (char)PTR_DAT_00104058[1];
        local_270 = local_270 + 1) {
602             local_258[local_270] = *(&strs)[local_270 % 9] ^ PTR_DAT_00104068[local_270] ^ 0x40;
603         }
604         local_237 = 0;
605         puts("Done.");
606         if (local_20 != *(long *) (in_FS_OFFSET + 0x28)) {
607             /* WARNING: Subroutine does not return */
608             __stack_chk_fail();
609         }
610         return 0;
611     }
612 }
613 }
614 }
615 }
616 }
617 iVar2 = decrypt((EVP_PKEY_CTX *)PTR_DAT_00104048, (uchar *)0x14, outlen_04, puVar8, in_R8);
618 printf("%s", CONCAT44(extraout_var_04, iVar2));
619 /* WARNING: Subroutine does not return */
620 exit(1);
621 }
622 }
```

I first open the gauntlet executable in ghidra and start looking at the main function. And find the following code snippet:

```

printf("%s", CONCAT44(extraout_var_07, iVar2));
printf("I assume you are mighty. But are you wise?\nI am going to decrypt the flag at %p,
but it will only reveal itself to those who are worthy.\n"
, local_258);
for (i = (long)key_4_1_; i < (char)PTR_DAT_00104058[1]; i = i + 1) {
    local_258[i] = *(&strs)[i % 9] ^ PTR_DAT_00104068[i] ^ 0x40;
}
local_237 = 0;
puts("Done.");

```

This is the decryption loop. It iterates over the encrypted bytes and XORs them against a repeating 9-byte key (strs) and the hex value 0x40. To solve this we can track down the raw bytes for the encrypted array (PTR_DAT_00104068), the 9-byte key (strs), and the length (PTR_DAT_00104058[1]) by double-clicking them in Ghidra.

Raw bytes for strs:

		DAT_00102008		XREF[2]:	main:00101622	
00102008	f3	??	F3h			59
00102009	3e	??	3Eh	>		59
0010200a	00	??	00h			60
0010200b	f2	??	F2h			60
0010200c	bf	??	BFh			60
0010200d	61	??	61h	a		60
0010200e	57	??	57h	W		60
0010200f	e8	??	E8h			60
00102010	f3	??	F3h			60
00102011	2b	??	2Bh	+		60
00102012	0a	??	0Ah			60
00102013	f8	??	F8h			61
00102014	00	??	00h			61

Raw bytes for the encrypted flag:

		DAT_0010205c		XREF[2]:	main:00101880
0010205c	88	??	88h		
0010205d	26	??	26h	&	
0010205e	1b	??	1Bh		
0010205f	ee	??	EEh		
00102060	fc	??	FCh		
00102061	3e	??	3Eh	>	
00102062	00	??	00h		
00102063	f2	??	F2h		
00102064	bb	??	BBh		
00102065	3c	??	3Ch	<	
00102066	13	??	13h		
00102067	f0	??	F0h		
00102068	fc	??	FCh		
00102069	27	??	27h	'	
0010206a	01	??	01h		
0010206b	bd	??	BDh		
0010206c	ba	??	BAh		
0010206d	27	??	27h	'	
0010206e	1c	??	1Ch		
0010206f	f8	??	F8h		
00102070	fc	??	FCh		
00102071	74	??	74h	t	
00102072	5b	??	5Bh	[	
00102073	97	??	97h		
00102074	00	??	00h		
00102075	00	??	00h		
00102076	00	??	00h		
00102077	00	??	00h		
		DAT_00102078		XREF[2]:	main:001018f0
-----	--	---			

Last byte:

00102049	00	??	00h		
		DAT_0010204a		XREF[2]:	main:00101840
0010204a	88	??	88h		
		DAT_0010204b		XREF[1]:	main:00101960

Now we have: key = [0xf3, 0xbb, 0xaf, 0xb0, 0xb0, 0x98, 0x88, 0x88, 0x88]

Then feed the raw bytes into Gemini where it gives you a script and shows the output:

Python

```
# The 33 bytes of the encrypted flag from Ghidra
ct = [
    0xd5, 0x88, 0x9a, 0xb3, 0xa4, 0x9e, 0xb3, 0xaa, 0xfb, 0xc7,
    0x8f, 0xdc, 0x82, 0xaf, 0xef, 0xa0, 0xfc, 0xa6, 0xec, 0x98,
    0x9d, 0x89, 0xc5, 0xac, 0xfc, 0xf9, 0x97, 0x87, 0x89, 0x82,
    0xc0, 0x82, 0xa5
]

# The array of 9 bytes pulled from the start of the 9 string pointers
key = [0xf3, 0xbb, 0xaf, 0xb0, 0xb0, 0x98, 0x88, 0x88, 0x88]

flag = ""

# Loop through all 33 bytes
for i in range(len(ct)):
    # The C code: local_258[local_270] = *(&strs)[local_270 % 9] ^ PTR_DAT_001040
    decrypted_byte = ct[i] ^ key[i % 9] ^ 0x40

    # Convert the hex byte back to an ASCII character
    flag += chr(decrypted_byte)

print(f"The flag is: {flag}")
```

If you run this in any Python environment (or just read the code closely), it spits out:

```
f5uCTF{b3tt3r_7h4n_cry5t41_4rm0r}
```

Flag: f5uCTF{b3tt3r_7h4n_cry5t41_4rm0r}

Upper Management

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$ ./upper_management
Management needs you to produce a flag.
I don't know what the spec is supposed to be, I'll just let you know if it looks wrong.
ok
Wrong. Next.
```

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$ get
Command 'get' not found, but there are 16 similar ones.
```

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$ strings upper_management
/lib64/ld-linux-x86-64.so.2
__libc_start_main
strcmp
puts
strlen
__isoc99_scanf
exit
__stack_chk_fail
libc.so.6
GLIBC_2.7
GLIBC_2.4
GLIBC_2.2.5
GLIBC_2.34
__gmon_start__
PTE1
H=X@@@
Wrong. Next.
Management needs you to produce a flag.
I don't know what the spec is supposed to be, I'll just let you know if it looks wrong.
%127s
Good enough, ship it.
:*3$"
GCC: (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0
.shstrtab
.interp
.note.gnu.property
.note.gnu.build-id
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.sec
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.got
.got.plt
.data
.bss
.comment
```

```
(kali㉿kali)-[~/CTFs/CTF_Class/reverseEngineering/hw1]
$ █
```

I first run strings on the executable to see if I can find any important information. In this case I do, I find that the output string, Management needs you to produce a flag, is stored in memory so that means I find it in ghidra so I do and I find the following code in what I believe to be the main function:

```
__isoc99_scanf("%127s", local_98);
sVar3 = strlen(local_98);
if (sVar3 == 0x26) {
    for (local_d0 = local_98; *local_d0 != '\0'; local_d0 = local_d0 + 1) {
        cVar1 = FUN_004011d6((int)*local_d0, (int)local_d0 - (int)local_98);
        *local_d0 = cVar1;
    }
    iVar2 = strcmp(local_98, local_c8);
    if (iVar2 == 0) {
        puts("Good enough, ship it.");
        uVar4 = 0;
        goto LAB_00401c23;
    }
}
```

I need to find local_c8 and figure out what FUN_004011d6 does in order to solve this:

```
char ciphering(char param_1, char param_2)
{
    if (('/' < param_1) && (param_1 < '~')) {
        return ((((((((((((((((((((((((((((((((param_1 + (param_2 + '\x03') * '\x13' ^
            (param_2 + '\x02') * '7' ^ param_2 + 2U) +
            (param_2 + '\x05') * '\x03' ^ (param_2 + '\x03') * '!' ^
            (param_2 + '\x02') * '-0xb) + (param_2 + '\x05') * '0' ^
            (param_2 + '\x03') * '-0x5e) + param_2 * '-0x71 + 0x8e ^
            (param_2 + '\x05') * '-0x44 ^ (param_2 + '\x01') * 's' +
            (param_2 + '\x02') * '-0x28 ^ (param_2 + '\x04') * '\x10' +
            (param_2 + '\x05') * '\x12' ^ (param_2 + '\x05') * '-0x35 ^
            (param_2 + '\x02') * '-0x18 ^ (param_2 + '\x01') * '-0xf) +
            param_2 * '?' + 0xc ^ (param_2 + '\x02') * '\b' ^
            (param_2 + '\x03') * ' ' + param_2 * '-0x6b + 0x84 ^
            (param_2 + '\x02') * '-0x2e) + param_2 * '-0x1c + 0x1e ^
            (param_2 + '\x02') * '-0x49 ^ (param_2 + '\x03') * '\x0f' ^
            (param_2 + '\x01') * 'm' + param_2 * '-5) - 10 ^
            (param_2 + '\x02') * '-0x77 ^ (param_2 + '\x02') * '9' + param_2 * 'j') -
            10 ^ (param_2 + '\x01') * '-0x4c ^ (param_2 + '\x02') * ']' + param_2 * 'G'
            + 0xac ^ (param_2 + '\x01') * ';' + param_2 * '\\\' + 0x16 ^
            (param_2 + '\x02') * '-0x66 ^ (param_2 + '\x03') * 'U' +
            (param_2 + '\x04') * '-8 ^ (param_2 + '\x01') * '\x05' +
            (param_2 + '\x05') * 'C' ^ (param_2 + '\x04') * '-0x52 ^
            (param_2 + '\x01') * '\n' ^ (param_2 + '\x01') * '\\\'
            (param_2 + '\x01') * '\x11' ^ (param_2 + '\x05') * '-0x14 ^
            (param_2 + '\x05') * '-0x2c) + (param_2 + '\x01') * 'F' ^
            (param_2 + '\x04') * 'c' ^ (param_2 + '\x01') * '-0x38) +
            (param_2 + '\x04') * '\x1d' ^ (param_2 + '\x02') * 'B' ^
            (param_2 + '\x01') * '-0x38) + param_2 * '\x03' + 0x44 ^ (param_2 + '\x03') * '-0x1c
            ) + (param_2 + '\x01') * '\x19' ^ (param_2 + '\x05') * '\v' +
            (param_2 + '\x05') * ' ' ^ (param_2 + '\x02') * '\x1c' + (param_2 + '\x01') * '-0xe
            ^ (param_2 + '\x03') * '-0x2b) + param_2 * 'H' + 0x96 ^ (param_2 + '\x05') * 'm' +
            (param_2 + '\x02') * '\f' ^ (param_2 + '\x01') * 'I' + param_2 * '\x14' + 0x2c ^
            (param_2 + '\x04') * '-6 ^ (param_2 + '\x01') * '\t' + param_2 * '-0x3c + -0x1c;
    }
    puts("Wrong. Next.");
    /* WARNING: Subroutine does not return */
    exit(1);
}
```

Woah. It does a lot, I don't think I am going to be able to repeat this process, but we will keep in mind.

```

local_10 = *(long *) (in_FS_OFFSET + 0x28);
local_c8[0] = -0x3e;
local_c8[1] = -0x26;
local_c8[2] = -0x23;
local_c8[3] = -0x7c;
local_c8[4] = 'd';
local_c8[5] = 'i';
local_c8[6] = 'C';
local_c8[7] = -0x5a;
local_c8[8] = '\x10';
local_c8[9] = -0x20;
local_c8[10] = '\t';
local_c8[0xb] = -0x22;
local_c8[0xc] = 'm';
local_c8[0xd] = -0xb;
local_c8[0xe] = 'C';
local_c8[0xf] = 'e';
local_c8[0x10] = -0x68;
local_c8[0x11] = '\x04';
local_c8[0x12] = -0x34;
local_c8[0x13] = -0x50;
local_c8[0x14] = '\x1d';
local_c8[0x15] = -0xd;
local_c8[0x16] = '~';
local_c8[0x17] = '3';
local_c8[0x18] = -0x27;
local_c8[0x19] = -0x4a;
local_c8[0x1a] = -0x31;
local_c8[0x1b] = '<';
local_c8[0x1c] = -0x38;
local_c8[0x1d] = -0x26;
local_c8[0x1e] = 's';
local_c8[0x1f] = '~';
local_c8[0x20] = 'd';
local_c8[0x21] = -9;
local_c8[0x22] = -0x22;
local_c8[0x23] = -0x5d;
local_c8[0x24] = -0x13;
local_c8[0x25] = -0x28;
local_c8[0x26] = 0;
puts(

```

This is what is stored in local_c8. I paste both pieces of information into Gemini and ask for the best solution. It came up with creating a python script that calls 0x4011d6 directly (the encryption function) for every possible character and then piece the flag together. Here is the script:

```

import gdb

target = [
    0xc2, 0xda, 0xdd, 0x84, 0x64, 0x69, 0x43, 0xa6,
    0x10, 0xe0, 0x09, 0xde, 0x6d, 0xf5, 0x43, 0x65,
    0x98, 0x04, 0xcc, 0xb0, 0x1d, 0xf3, 0x7e, 0x33,
    0xd9, 0xb6, 0xcf, 0x3c, 0xc8, 0xda, 0x73, 0x7e,
    0x64, 0xf7, 0xde, 0xa3, 0xed, 0xd8
]

gdb.execute("set pagination off")
# Use starti to pause at the very first instruction instead of main
gdb.execute("starti")

flag = ""
print("\n[*] Brute-forcing the flag...")

for i in range(38):
    for c in range(48, 126):
        try:
            res = gdb.parse_and_eval(f"((char (*)(int, int))0x4011d6)({c}, {i})")

            if (int(res) & 0xff) == target[i]:
                flag += chr(c)
                print(f"Index {i:02}: Found '{chr(c)}'")
                break
        except Exception as e:
            pass

print(f"\n[*] Recovered Flag: {flag}")
gdb.execute("quit")
~

```

And it gave:

```
Program stopped.  
0x00007ffff7fdfa80 in ?? () from /lib64/ld-linux-x86-64.so.2
```

```
[*] Brute-forcing the flag...
```

```
Index 00: Found 'f'  
Index 01: Found 's'  
Index 02: Found 'u'  
Index 03: Found 'C'  
Index 04: Found 'T'  
Index 05: Found 'F'  
Index 06: Found '{'  
Index 07: Found '5'  
Index 08: Found 't'  
Index 09: Found 'i'  
Index 10: Found '1'  
Index 11: Found '1'  
Index 12: Found '-'  
Index 13: Found 'b'  
Index 14: Found '3'  
Index 15: Found 't'  
Index 16: Found 't'  
Index 17: Found 'e'  
Index 18: Found 'r'  
Index 19: Found '-'  
Index 20: Found '7'  
Index 21: Found 'h'  
Index 22: Found '4'  
Index 23: Found 'n'  
Index 24: Found '-'  
Index 25: Found '7'  
Index 26: Found '3'  
Index 27: Found 'c'  
Index 28: Found 'h'  
Index 29: Found '-'  
Index 30: Found 's'  
Index 31: Found 'u'  
Index 32: Found 'p'  
Index 33: Found 'p'  
Index 34: Found '0'  
Index 35: Found 'r'  
Index 36: Found '7'  
Index 37: Found '}'
```

```
[*] Recovered Flag: fsuCTF{5ti11_b3ttter_7h4n_73ch_supp0r7}  
A debugging session is active.
```

```
Inferior 1 [process 48218] will be killed.
```

Flag: fsuCTF{5ti11_b3ttter_7h4n_73ch_supp0r7}